TECHNICAL ARCHITECTURE & CODEBASE MASTER GUIDE

# DocIntel — AI Document Intelligence Platform

An exhaustive multi-tier engineering handbook covering Database Schemas, Python OCR/AI Microservice, Node.js Express Gateway, React Single-Page Application, Server Startup Runbooks, and Synchronized Business Logic.

| | |
|---|---|
| **Document Version:** | v4.8 Multi-Tier Enterprise Architecture |
| **Target Stack:** | Node.js 22 + Python 3.12 + FastAPI + React 18 + MySQL 8.0 |
| **Platform Scope:** | OCR Ingestion, Entity Extraction, Compliance & Audit Trail |
| **Date of Compilation:** | October 2026 |
| **Author / System:** | DocIntel Core Engineering & Systems Architecture |

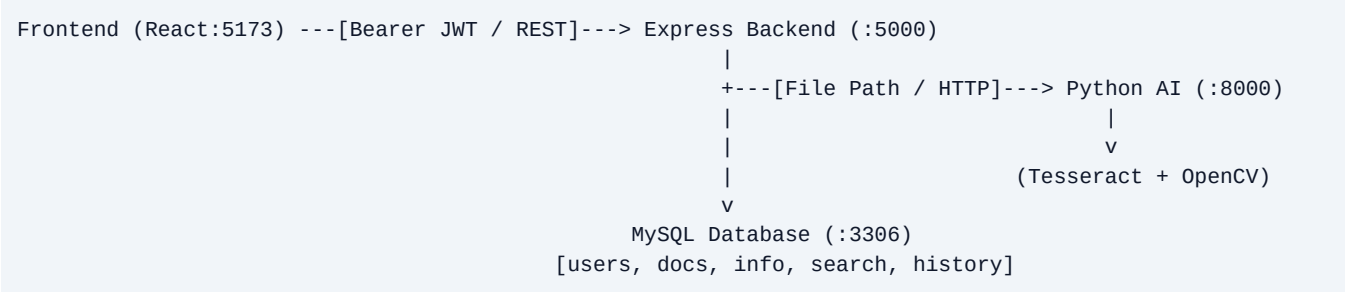

| SECTION | TOPIC COVERAGE |
|---|---|
| Section 1 | Executive Architecture Overview & Component Topology |
| Section 2 | Server Orchestration & Startup Runbook (All 4 Tiers) |
| Section 3 | Database Layer Breakdown & Relational Schema (database/) |
| Section 4 | Python OCR & AI Intelligence Engine (python-service/) |
| Section 5 | Node.js Express API Gateway & Business Layer (backend/) |
| Section 6 | Frontend Single-Page Application (frontend/) |
| Section 7 | Complete REST API Endpoints Specification |
| Section 8 | End-to-End Ingestion Lifecycle & Data Flow |
| Section 9 | Key Algorithms, Synchronized Metrics & Problem-Solving Ledger |

# 1. Executive Architecture Overview

DocIntel is a centralized company document intelligence platform designed to eliminate document fragmentation across desktop folders, email threads, and local drives. It implements a decoupled multi-tier architecture where each service has strict boundaries of responsibility:

- **Frontend Tier (Port 5173)**: Single-Page Application written in React 18, Vite, and modern CSS. Manages UI states, authenticated JWT sessions, document previews, and reactive dashboards.
- **API Gateway Tier (Port 5000)**: Express.js server providing authentication (JWT + bcrypt), authorization guards (Role-Based Access Control), disk storage handling via Multer, audit logging, and parameterized relational MySQL communication.
- **AI Microservice Tier (Port 8000)**: Python 3.12 FastAPI microservice. Executes optical character recognition (OCR) with Tesseract, computer vision image preprocessing via OpenCV, document classification, regex parsing for statutory entities (GSTIN, PAN, dates, financial amounts), and executive summary synthesis.
- **Database Tier (Port 3306)**: MySQL 8.0 relational database schema (docintel) maintaining ACID compliance, foreign key constraints, soft-deletion attributes, and full audit histories.

### High-Level Topology Representation:




# 2. Server Orchestration & Startup Runbook

DocIntel requires four execution environments running in tandem. Below are the precise startup commands and health checks for each service:

| SERVICE | PORT | DIRECTORY & START COMMAND | HEALTH VERIFICATION |
|---|---|---|---|
| **1. MySQL DB** | 3306 | net start MySQL80 (or via Windows Services / XAMPP) | SELECT 1; on database docintel |
| **2. Python AI** | 8000 | cd python-service<br>.\venv\Scripts\Activate.ps1<br>uvicorn app:app --host 0.0.0.0 --port 8000 --reload | GET http://localhost:8000/health<br>→ 200 OK Status |
| **3. Node.js API** | 5000 | cd backend<br>npm run dev | GET<br>http://localhost:5000/api/health<br>→ 200 OK Status |
| **4. React Web** | 5173 | cd frontend<br>npm run dev | Open browser at<br>http://localhost:5173 |

# 3. Database Architecture (database/)

The database is structured to support strong referential integrity, cascading updates, high-speed search indexing, and complete chronological audit tracking.

## Database File Index:

- **database/schema.sql**: Full DDL script defining tables, foreign keys, and indexes.
- **database/seed.sql**: Realistic demo data containing admin, users, documents, and extracted metadata.
- **database/user_migration.sql**: Migration script adding department and designation columns to users.
- **database/migrate_add_dept.sql**: Adds soft-delete attributes (is_deleted, deleted_at, deleted_by).
- **database/run_migration.js**: Automated Node.js migration execution runner.

## Core Relational Tables Schema:

### Table 1: users

| Field Name | Type & Attributes | Description |
|---|---|---|
| id | INT AUTO_INCREMENT PRIMARY KEY | Unique user ID |
| name | VARCHAR(100) NOT NULL | Full display name |
| email | VARCHAR(255) NOT NULL UNIQUE | Login credential (indexed) |
| password_hash | VARCHAR(255) NOT NULL | Bcrypt salted hash |
| role | ENUM('USER', 'ADMIN') DEFAULT 'USER' | RBAC authorization level |
| department | VARCHAR(100) | Corporate department |
| designation | VARCHAR(100) | Job title / position |
| created_at / updated_at | DATETIME | Automatic audit timestamps |

### Table 2: documents

| Field Name | Type & Attributes | Description |
|---|---|---|
| id | INT AUTO_INCREMENT PRIMARY KEY | Document identifier (DOC-0001) |
| uploaded_by | INT NOT NULL | Foreign key references users(id) |
| original_filename | VARCHAR(255) NOT NULL | Source file name |
| stored_filename | VARCHAR(255) NOT NULL | Disk storage filename |
| file_path | VARCHAR(500) NOT NULL | Path on filesystem (uploads/...) |
| file_type | VARCHAR(50) | MIME type (PDF, PNG, JPEG) |
| file_size | INT | File size in bytes |
| document_type | ENUM('Invoice', 'Quotation', 'Tender', ...) | Document business category |
| status | ENUM('PENDING', 'DONE') | Workflow approval status |
| processing_status | ENUM('UPLOADED', 'PROCESSING', ...) | AI OCR pipeline state |
| document_date / due_date | DATE | Issue and deadline cutoff dates |
| is_deleted | TINYINT(1) DEFAULT 0 | Soft-deletion flag (0=Active, 1=Recycled) |

### Table 3: document_information (1-to-1 with documents)

| Field Name | Type & Attributes | Description |
|---|---|---|
| id | INT AUTO_INCREMENT PRIMARY KEY | Metadata entry ID |
| document_id | INT NOT NULL UNIQUE | FK to documents(id) ON DELETE CASCADE |
| company_name | VARCHAR(255) | Extracted vendor/issuer entity |
| gst_number | VARCHAR(50) | Indian GST statutory number |
| document_number | VARCHAR(100) | Invoice/PO/Tender reference # |
| document_date / due_date | DATE | Extracted issue and deadline dates |
| amount | DECIMAL(15,2) | Parsed financial total |
| key_points | TEXT | Extracted bullets / line items |
| summary | TEXT | Synthesized executive summary paragraph |
| extracted_text | LONGTEXT | Full raw OCR text transcript |

### Table 4 & 5: search_history & document_history

| Field Name | Type & Attributes | Description |
|---|---|---|
| search_history | id, user_id, search_query, filters, result_count, searched_at | Tracks query queries and result counts |
| document_history | id, user_id, document_id, action, details, created_at | Audit ledger: VIEW, UPLOAD, SEARCH, EDIT_META, DELETE |

# 4. Python OCR & AI Intelligence Engine (python-service/)

The Python service is an asynchronous microservice written in FastAPI. It handles heavy computer vision and natural language processing workloads, isolating them from the Node.js API Gateway.

## app.py — *FastAPI Service Entrypoint*

Configures CORS middleware to allow requests from the Node.js backend. Defines the health check endpoint GET /health and the primary ingestion endpoint POST /process. Receives the file path, calls the processing pipeline, handles processing timeouts and exceptions, and returns standard JSON payloads.

## processing/ocr_processor.py — *Pipeline Conductor & Orchestrator*

The master controller for document processing. Coordinates the workflow: (1) invokes text extraction, (2) passes raw text to the classifier to determine document type, (3) executes entity extraction for structured key-values, and (4) bundles the response with processing confidence scores and timestamps.

## processing/text_extractor.py — *Multi-Format Ingestion & Computer Vision*

Ingests PDFs and image files (PNG, JPG, TIFF). For digital PDFs, it leverages pdfplumber and PyPDF2 for rapid extraction. If character density is low (< 50 chars), it falls back to page rendering and OCR. For images, it utilizes OpenCV (cv2) for image optimization: converting to grayscale, applying Gaussian bilateral filtering to reduce noise while preserving edges, and applying adaptive Otsu thresholding before passing to pytesseract.image_to_string(). Includes automated binary path detection for Windows.

## processing/classifier.py — *Rule-Based Document Classification*

Uses weighted keyword dictionaries and regular expression pattern density to classify documents into standard business types (Invoice, Quotation, Tender, Purchase Document, Agreement, Other). Computes score differentials to ensure high-precision classification.

## processing/information_extractor.py — *Entity & Key-Value Extraction Engine*

Applies regular expressions and structural heuristics to extract critical business entities:
• **GST Number**: \b[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}\b
• **PAN Number**: \b[A-Z]{5}[0-9]{4}[A-Z]{1}\b
• **Company Name**: Contextual header extraction preceding addresses, GSTINs, or lines following 'M/s'.
• **Dates & Deadlines**: Multi-pattern date parsing (DD/MM/YYYY, YYYY-MM-DD, DD-Mon-YYYY) distinguishing document date from due date.
• **Financial Amounts**: Currency-aware numeric parsing supporting commas, decimals, INR, USD, and EUR.
• **Summary Generator**: Synthesizes a structured 3-sentence executive summary.

# 5. Node.js Express API & Business Layer (backend/)

The Node.js backend operates as the central API Gateway. It enforces security, orchestrates business services, handles file uploads, communicates with MySQL, and coordinates with the Python AI service.

## src/server.js — *Server Entrypoint & Middleware Assembly*

Initializes Express, enables CORS for the React frontend, configures body-parsers, mounts static file serving for uploads, binds modular routers under /api, and attaches global error handling middleware on port 5000.

## src/config/db.js — *MySQL Connection Pool*

Configures an asynchronous connection pool using mysql2/promise with a connection limit of 10. Exports the pool for parameterized queries, preventing SQL injection.

## src/middleware/authMiddleware.js & roleMiddleware.js — *Security & RBAC Guards*

authMiddleware inspects the Authorization: Bearer <token> header, verifies the JWT signature with JWT_SECRET, and attaches { userId, email, role } to req.user. roleMiddleware verifies whether req.user.role === 'ADMIN', returning 403 Forbidden for unauthorized access.

## src/middleware/uploadMiddleware.js — *Multer Multipart Ingestion*

Configures disk storage destination and generates unique UUID filenames. Enforces a 25MB file size limit and validates MIME types (PDF, PNG, JPEG).

## src/services/documentService.js — *Core Business Queries & Statistics Engine*

Implements comprehensive document lifecycle operations:
• **getStats()**: Calculates synchronized repository metrics across tables: Total Volume, Action Queue (Pending), Upcoming Pipeline (due within 14 days), Overdue Documents (past due date and not DONE), and Purge Queue.
• **getDocuments()**: Dynamic SQL query builder supporting pagination, search strings, status filters, and sorting.
• **updateDocument()**: In-line metadata updates committing changes to both documents and document_information.
• **deleteDocument()**: Performs safe soft-deletions (is_deleted=1) with audit logging.

## src/services/pythonProcessingService.js — *Node-to-Python Microservice Bridge*

Asynchronously dispatches document file paths to Python POST http://localhost:8000/process. On response, updates document_information via SQL UPSERT and marks the document as PROCESSED.

## src/controllers/historyController.js — *Audit & Search Tracking Controller*

• **getSearchHistory**: Returns user query history with automatic seeding fallback.
• **addSearchHistory**: Deduplicates repeated searches by updating timestamps, avoiding database clutter.
• **getActivityHistory**: Queries document_history enriched with company names and user roles.

# 6. Frontend Single-Page Application (frontend/)

Built with React 18, Vite, and custom CSS variables, the frontend provides a dark-navy corporate design optimized for high-density document intelligence and responsive user feedback.

## Dashboard.jsx — *Executive Operational Overview*

Displays 4 core stat cards (TOTAL VOLUME, ACTION QUEUE, UPCOMING PIPELINE, URGENT ATTENTION). Renders the recent documents table with live tab filters (All, Pending, Upcoming, Done). Directly connects to document upload and detail inspection views.

## DocumentDates.jsx — *Chronological Milestone & Compliance Ledger*

Features 3 top stat cards (UPCOMING DEADLINES, PENDING & SCHEDULED, OVERDUE DOCUMENTS). Presents a 3-column chronological Kanban board:
• **Column 1**: Upcoming Deadlines (Deadlines in 1 to 2 weeks).
• **Column 2**: Pending & Scheduled (Documents pending review or future milestones).
• **Column 3**: Recently Due / Overdue (Past cutoff dates requiring immediate resolution).

## DocumentDetails.jsx — *Split-Screen Inspection & In-Line Metadata Editor*

Displays an embedded document viewer (PDF/Image) side-by-side with AI-extracted attributes. Features editable fields (Company, GSTIN, Due Date, Amount, Status Toggle) and a **Save to Database** button that persists modifications directly to MySQL via PUT /api/documents/:id.

## History.jsx — *1-Screen Executive Console & Audit Explorer*

Contained in a single screen viewport:
• **Left Column**: Recent searches with quick-search bar and 1-click execution chips.
• **Right Column**: Recent activity showing distinct, condensed business events (view counts grouped).
• **Audit Trail Modal**: Full searchable audit log modal with category filters (All, Edits, Views, Uploads, Searches).

## AdminManagement.jsx — *Administrator Control Console*

Protected administrative interface with 4 synchronized stat cards (INDEXED FILES, PENDING AUDIT, OVERDUE ESCALATIONS, VERIFIED / DONE). Includes document management table, metadata drawer, soft delete, and CSV export.

## Navbar.jsx — *Global Header & SLA Notification Dropdown*

Features real-time engine status indicators and an active notification bell. Displays red badge counts for overdue documents and provides a slide-down notification menu with direct links to affected records.

## AuthContext.jsx & api.js — *Authentication State & Axios Client*

Persists JWT tokens and user profiles in localStorage. Centralized Axios client automatically injects Authorization: Bearer <token> headers on all outgoing API requests.

# 7. Complete REST API Endpoints Specification

All API endpoints follow standard REST conventions and return structured JSON responses: { success: boolean, message: string, data: object }.

| METHOD | ENDPOINT | AUTH | DESCRIPTION |
|---|---|---|---|
| POST | /api/auth/register | Public | Register new account with name, email, password, role |
| POST | /api/auth/login | Public | Verify credentials and issue signed 7-day JWT token |
| GET | /api/auth/me | Bearer JWT | Fetch authenticated user profile and permissions |
| GET | /api/documents | Bearer JWT | Retrieve paginated documents with search and filters |
| POST | /api/documents/upload | Bearer JWT | Upload file (PDF/Image) and trigger Python AI pipeline |
| GET | /api/documents/stats | Bearer JWT | Fetch unified repository statistics (total, pending, overdue) |
| GET | /api/documents/:id | Bearer JWT | Fetch complete document record and extracted AI attributes |
| PUT | /api/documents/:id | Bearer JWT | Update metadata in database and record EDIT_META audit |
| DELETE | /api/documents/:id | Admin JWT | Soft-delete document (is_deleted=1) with audit logging |
| GET | /api/history/search | Bearer JWT | Retrieve recent search queries for current user |
| POST | /api/history/search | Bearer JWT | Log search query and result count with deduplication |
| GET | /api/history/activity | Bearer JWT | Fetch repository activity history and audit ledger |
| GET | /api/users | Admin JWT | List all registered users in the organization |
| POST | /process (Python :8000) | Internal | Process file via Tesseract OCR and extract structured keys |

# 8. End-to-End Data Flow & Ingestion Lifecycle

**Step 1: Intake & Upload** — User uploads a document (e.g., Invoice PDF or PO scan). The React frontend dispatches a multipart request to Express.

**Step 2: File Persistence** — Multer writes the file to backend/uploads/, generating a secure UUID filename. A record is inserted into documents with status='PENDING' and processing_status='UPLOADED'.

**Step 3: Asynchronous AI Trigger** — Node.js dispatches an HTTP request to the Python microservice (POST http://localhost:8000/process) passing the file path.

**Step 4: Computer Vision & OCR** — Python applies OpenCV preprocessing (grayscale, bilateral filtering, adaptive thresholding) and executes Tesseract OCR / pdfplumber extraction.

**Step 5: Classification & Extraction** — The classifier determines document type. Heuristics extract GSTIN, dates, due date, amounts, and summary.

**Step 6: Database Persistence** — Python returns JSON to Node.js. Node executes an UPSERT into document_information and marks processing_status='PROCESSED'.

**Step 7: User Review & Persistence** — User inspects the extracted data in DocumentDetails, edits any fields, and clicks 'Save to Database', updating MySQL and setting status='DONE'.

# 9. Key Algorithms, Synchronized Metrics & Problem-Solving Ledger

## Unified Overdue Metric Calculation

**Problem**: Dashboard showed 0 overdue documents while the top notification banner showed 2 overdue documents and Document Dates showed 2.
**Solution**: Unified calculation in backend documentService.js:
WHERE d.is_deleted = 0 AND ((COALESCE(d.due_date, di.due_date) < CURDATE() AND d.status != 'DONE') OR d.status = 'FAILED').
Ensures completed documents (status='DONE') are never counted as overdue blockers.

## Upcoming Deadlines Window (1–2 Weeks)

**Problem**: Conflicting ranges across pages (Due Next 7 Days vs 1-2 Weeks).
**Solution**: Standardized on a 14-day window: active documents where due date falls between CURDATE() and CURDATE() + 14 DAYS.

## Search Logging & Throttled Activity Deduplication

**Problem**: Repeated search queries and rapid page reloads created spam in the activity feed.
**Solution**: Implemented deduplication in historyController.js that updates existing timestamps on repeated queries, and throttled view tracking within 5-minute windows.

## Case-Sensitivity Fix on Admin Console

**Problem**: Pending Audit showed 0 because of case-sensitive comparison (d.status === 'pending' vs 'PENDING' in MySQL).
**Solution**: Updated to case-insensitive comparison (d.status?.toLowerCase() === 'pending'), correctly displaying 2.